COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S4
DATE PERFORMED:	July 31, 2026	**DATE SUBMITTED:**	July 31, 2026
NAME:	Clifford Lee. C. Doplayna	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. _4.1_

1. PROCEDURE OUTPUT

1.**ILO A:** Implement a stack using the C++ STL (`std::stack`).

```cpp
#include <iostream>
#include <stack>

int main(){

std::stack<char> stack1;

std::cout << "Is the stack empty? " << stack1.empty() << std::endl;
stack1.push('c');
std::cout << "current top of the stack: " << stack1.top() << std::endl;
stack1.push('l');
std::cout << "current top of the stack: " << stack1.top() << std::endl;
stack1.push('i');
std::cout << "current top of the stack: " << stack1.top() << std::endl;
stack1.push('f');

std::stack<char> stack2;

std::cout << "current top of the stack: " << stack1.top() << std::endl;
stack1.push('f');

std::cout << "Is the stack empty? " << stack1.empty() << std::endl;
std::cout << "What is the size of the stack? " << stack1.size() << std::endl;

return 0;
}
```

```
Is the stack empty? 1
current top of the stack: c
current top of the stack: l
current top of the stack: i
current top of the stack: f
current top of the stack: f
Is the stack empty? 0
What is the size of the stack? 6
PS C:\Users\TIPQC\OneDrive\Documents\hoa4p1\ILOA>
```

2.**ILO B.1:** Implement a stack using a Static Array.

```cpp
#include<iostream>

//global variables:
#define maxSize 10
int stackArr[maxSize];
int top = -1, newData;

//function prototypes:
// Top, Push , Pop, isEmpty, isFul

//observer operators
bool isEmpty();
bool isFull();
void Top(); // the T is capital
void displayAll(); // goal is to display the stack following the LIFO principle

//transformer operators
void push();
void pop();


int main(){
    int choice;
    while(true){
        std::cout<<"-----------------------------\n";
        std::cout<<" Stack operations: "<<std::endl;
        std::cout<<" [1] push [2] pop [3] isEmpty [4] isFull [5] Top"<<std::endl;
        std::cin >> choice;
        std::cout<<"-----------------------------\n";
        switch(choice){
            case 1: push();
            break;
            case 2: pop();
            break;
            case 3: std::cout<<"is the stack empty? "<< isEmpty()<<std::endl;
            break;
            case 4:std::cout<<"is the stack full? "<< isFull()<<std::endl;
            break;
            case 5: Top();
            break;
            default: std::cout<<"Invalid Choice! "<<std::endl;
            break;
        }
    }
```

```
45      return 0;
46  }
47
48
49  bool isEmpty(){
50      return (top == -1);
51  }
52
53  bool isFull(){
54      return (top == maxSize-1);
55  }
56
57  void Top(){
58      if(isEmpty()) return;
59
60      std::cout<<"The top element is: "<< stackArr[top]<<std::endl;
61  }
62
63  void push(){
64      if (isFull()) return;
65
66      std::cout<<"what character do you want to push? ";
67      std::cin >> newData;
68
69      stackArr[++top] = newData;
70
71  }
72
73  void pop(){
74      if(isEmpty()) return;
75
76      std::cout<<"popping: "<< stackArr[top] <<std::endl;
77
78      top --;
79
80
81  }
```

```
 [1] push [2] pop [3] isEmpty [4] isFull [5] Top
1
==============================
what character do you want to push? 2
==============================
 Stack operations:
 [1] push [2] pop [3] isEmpty [4] isFull [5] Top
1
==============================
what character do you want to push? 1
==============================
 Stack operations:
 [1] push [2] pop [3] isEmpty [4] isFull [5] Top
1
==============================
what character do you want to push? 2
==============================
 Stack operations:
 [1] push [2] pop [3] isEmpty [4] isFull [5] Top
1
==============================
what character do you want to push? 3
==============================
 Stack operations:
 [1] push [2] pop [3] isEmpty [4] isFull [5] Top
3
==============================
is the stack empty? 0
==============================
 Stack operations:
 [1] push [2] pop [3] isEmpty [4] isFull [5] Top
4
==============================
is the stack full? 0
==============================
 Stack operations:
 [1] push [2] pop [3] isEmpty [4] isFull [5] Top
5
==============================
The top element is: 3
==============================
 Stack operations:
 [1] push [2] pop [3] isEmpty [4] isFull [5] Top
```

2. CODE ANALYSIS

1.The program shows how elements are added sequentially ('c', 'l', 'i', 'f', and 'f') while checking the top value at each stage by creating a C++ character stack using std::stack. Finally, it outputs the stack's final total size of five elements and confirms that the stack is not empty.

2.This program uses C++ to implement a unique array-based stack data structure with a fixed maximum size of 10. The user can continuously push and pop elements, check if the stack is full or empty, and view the current top item from an interactive menu that appears inside an infinite loop.

B.2. Stacks using Linked Lists

- Modify the code given above to include a function that will display all elements in the stack.
- Provide a description of each operation provided.
- Include your output in the output section of your activity template

ILOAB2 > C stacklist.h > ...

```cpp
#ifndef STACK_LIST_H
#define STACK_LIST_H

#include <iostream>

// Class Node that contains data and pointer to the next node
template <typename T>
class Node {
public:
    T data;     // Value to store
    Node* next; // Pointer to the next node
};

// Global pointer for the top of the stack (Requires C++14 or newer)
template <typename T>
Node<T>* head = nullptr;

// --- Stack Operations ---

// PUSH: Inserts an element onto the top of the stack
template <typename T>
void push(T newData) {
    Node<T>* newNode = new Node<T>;
    newNode->data = newData;
    newNode->next = head<T>;
    head<T> = newNode;
}

// POP: Removes and returns the top element from the stack
template <typename T>
T pop() {
    if (head<T> == nullptr) {
        std::cout << "Stack underflow.\n";
        return T(); // Returns default value if empty to prevent crash
    }

    Node<T>* temp = head<T>;
    T tempVal = temp->data;

    head<T> = head<T>->next;
```

```
41      delete temp;
42      return tempVal;
43  }
44
45  // IS EMPTY: Returns true if the stack has no elements
46  template <typename T>
47  bool isEmpty() {
48      return head<T> == nullptr;
49  }
50
51  // TOP: Displays the value at the top of the stack without removing it
52  template <typename T>
53  void Top() {
54      if (isEmpty<T>()) {
55          std::cout << "Stack underflow\n";
56          return;
57      }
58      std::cout << "Top of Stack: " << head<T>->data << std::endl;
59  }
60
61  // DISPLAY ALL: Traverses and prints all elements from top to bottom
62  template <typename T>
63  void displayALL() {
64      if (head<T> == nullptr) {
65          std::cout << "Stack is empty (underflow).\n";
66          return;
67      }
68      std::cout << "Displaying the stack:\n";
69      Node<T>* temp = head<T>;
70      while (temp != nullptr) {
71          std::cout << temp->data << "\n";
72          temp = temp->next;
73      }
74      std::cout << "\n";
75  }
76
77  #endif
78
```

ILOAB2 > main.cpp > main()

```cpp
#include <iostream>
#include "stackList.h"

int main() {
    std::cout << "--- Testing Stack Operations ---\n\n";

    // Push elements onto the stack
    std::cout << "Pushing 10, 20, 30...\n";
    push(10);
    push(20);
    push(30);

    // Display top element and all elements
    Top<int>();
    displayALL<int>();

    // Pop an element
    std::cout << "Popping top element: " << pop<int>() << "\n\n";

    // Display stack state after pop
    Top<int>();
    displayALL<int>();

    // Test isEmpty
    std::cout << "Is stack empty? " << (isEmpty<int>() ? "Yes" : "No") << "\n";

    return 0;
}
```

```
--- Testing Stack Operations ---

Pushing 10, 20, 30...
--- Testing Stack Operations ---

Pushing 10, 20, 30...

Pushing 10, 20, 30...
Top of Stack: 30
Pushing 10, 20, 30...
Top of Stack: 30
Displaying the stack:
Top of Stack: 30
Displaying the stack:
Displaying the stack:
30
20
10
30
20
10
20
10
10


Popping top element: 30

Top of Stack: 20
Displaying the stack:
20
10

Is stack empty? No
PS C:\Users\TIPQC\OneDrive\Documents\HOA4P1STACK>
```

push(T newData)
It adds a new element to the top of the stack. It dynamically allocates a new node containing newData, sets its next pointer to the current head, and updates head to point to this new node.

pop()
It removes and returns the top element of the stack. It checks for stack underflow, saves the current head node, advances head to head->next, frees the memory allocated for the removed node using delete, and returns the stored data.

isEmpty()
It checks whether the stack contains any elements. It returns true if head is nullptr, and false otherwise.

Top()
It views the value at the top of the stack without removing it. It checks if the stack is empty to avoid errors, and prints the data stored in the current head node.

displayALL() (Requested Task)
It prints all elements currently in the stack from top to bottom. It checks if the stack is empty first, then iterates through the linked list using a temporary pointer until it reaches nullptr.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Expression	Valid? (Y/N)	Output (Console Screenshot)	• Analysis
(A+B)+(C-D)			
((A+B)+(C-D)			
((A+B)+[C-D])			
((A+B]+[C-D]}			

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini Ai, Quillbot**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Extra information, Grammar checker**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

COMPUTER ENGINEERING

QUEZON CITY

Extent of Use: **I used the gemini ai to get more information about stacks and how I can apply it to the program. I used the quillbot for grammar checker for my answer in conclusion part.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.

Conclusion

Assessment Rubric

T.I.P. S.O. 7 Rubric for Lifelong Learning

Criteria	Ratings						Pts
Acquire and apply new knowledge from outside sources	6 pts **Excellent** Educational interests and pursuits exist and flourish outside classroom requirements, knowledge and/or experiences are pursued independently and applies knowledge learned into practice	5 pts **Good** Educational interests and pursuits exist and flourish outside classroom requirements, knowledge and/or experiences are pursued independently	4 pts **Satisfactory** Look beyond classroom requirements, showing interest in pursuing knowledge independently	3 pts **Unsatisfactory** Begins to look beyond classroom requirements, showing interest in pursuing knowledge independently	2 pts **Poor** Relies on classroom instruction only	1 pts **Very Poor** No initiative or interest in acquiring new knowledge	6 pts
Learn independently	6 pts **Excellent** Completes an assigned task independently and practices continuous improvement	5 pts **Good** Completes an assigned task without supervision or guidance	4 pts **Satisfactory** Requires minimal guidance to complete an assigned task	3 pts **Unsatisfactory** Requires detailed or step-by-step instructions to complete a task	2 pts **Poor** Shows little interest to complete a task independently	1 pts **Very Poor** No interest to complete a task independently	6 pts
Critical thinking in the broadest context of technological change	6 pts **Excellent** Synthesizes and integrates information from a variety of sources; formulates a clear and precise perspective; draws appropriate conclusions	5 pts **Good** Evaluate information from a variety of sources; formulates a clear and precise perspective	4 pts **Satisfactory** Analyze information from a variety of sources; formulates a clear and precise perspective.	3 pts **Unsatisfactory** Apply the gathered information to formulate the problem	2 pts **Poor** Gather and summarized the information from a variety of sources but failed to formulate the problem	1 pts **Very Poor** Gather information from a variety of sources	6 pts
Creativity and adaptability to new and emerging technologies	6 pts **Excellent** Ideas are combined in original and creative ways in line with the new and emerging technology trends to solve a problem or address an issue.	5 pts **Good** Ideas are creative and adapt the new knowledge to solve a problem or address an issue	4 pts **Satisfactory** Ideas are creative in solving a problem or address an issue	3 pts **Unsatisfactory** Shows some creative ways to solve the problem	2 pts **Poor** Shows initiative and attempt to develop creative ideas to solve the problem	1 pts **Very Poor** Ideas are copied or restated from the sources consulted	6 pts
							Total Points: 24